Code No: 115AP **R13**

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY HYDERABAD
B. Tech III Year I Semester Examinations, August/September - 2024
COMPILER DESIGN
(Computer Science and Engineering)

Time: 3 hours **Max. Marks: 75**

Note: i) Question paper consists of Part A, Part B.
ii) Part A is compulsory, which carries 25 marks. In Part A, Answer all questions.
iii) In Part B, Answer any one question from each unit. Each question carries 10 marks and may have a, b as sub questions.

Used Paper Aug-2024

PART - A

(25 Marks)

1.a)	What is the primary goal of the lexical analysis phase in a compiler?	[2]
b)	What does LL(1) stand for in LL(1) parsing, and what does the '1' represent?	[3]
c)	What is an ambiguous grammar, and why can it pose a problem for parsers?	[2]
d)	What does LALR stand for, and how does it improve upon LR parsing?	[3]
e)	What is the role of a type checker in semantic analysis?	[2]
f)	What information does a symbol table entry usually contain for a variable?	[3]
g)	What are the main factors to consider when deciding on code optimizations?	[2]
h)	How does copy propagation work, and what is its main benefit in code optimization?	[3]
i)	What is object code, and why is it important in the compilation process?	[2]
j)	How does register allocation impact the performance of a compiled program?	[3]

PART - B

(50 Marks)

2. Compare and contrast regular grammar and regular expressions. How are they used in the context of lexical analysis? Provide examples of how regular expressions can be applied to common programming language features. [10]

OR

3.a) Discuss the process of top-down parsing. How does it use context-free grammars to parse input strings?
b) Explain what backtracking means in the context of top-down parsing. [5+5]

4. Explain the concept of look ahead in LR parsing. How does the look ahead symbol influence the parsing decisions, and what role does it play in constructing the parsing table? [10]

OR

5.a) Discuss the differences between LR parsing and LL parsing.
b) With suitable example, explain how to construct predictive parser table. [4+6]

6. What is a type checker in the context of semantic analysis? How does a type checker ensure that operations in a program are semantically correct with respect to the data types involved? [10]

OR

7. Explain stack and heap storage allocation strategies and mention their merits and demerits. [10]

8. Describe the Directed Acyclic Graph (DAG) representation in the context of code optimization. How does DAG representation help in optimizing expressions and managing intermediate code? [10]

OR

9. Explain Redundant Subexpression Elimination (RSE). How does this optimization technique work, and what is its impact on program performance? Provide an example of how redundant subexpressions are identified and eliminated. [10]

10. Describe the process of register allocation and assignment in code generation. Why is register allocation crucial for efficient code execution, and what are the common algorithms used for allocating registers? [10]

OR

11. Describe the role of machine-dependent optimizations in enhancing the performance of generated object code. How do these optimizations leverage specific features of the target architecture? [10]

---ooOoo---

Used Paper Aug-2024